Dubai DLD Monthly Transaction Pipeline

Technical Documentation & Operational Guide

1. Pipeline Overview

The pipeline functions as a single, combined ETL (Extract, Transform, Load) script (dld_pipeline.py) that processes raw monthly transactional data from the Dubai Land Department (DLD) and converts it into a normalized, enriched Excel spreadsheet. This spreadsheet is structured and validated for direct loading into the database tables.

The script performs raw CSV loading, duplicate removal, date/quarter parsing, property type normalization, database-backed project/location enrichment, dynamic indexing (assigning sequential nrXXXX values to new project-location combinations), coordinate fallbacks, and schema reordering.

2. Configuration Settings

All parameters and connections are specified in the CONFIG section of the pipeline script:

Parameter	Type	Default Value	Description
RAW_CSV_PATH	Path	D:\Database\...\1_july_to_12_aug_raw.csv	Input raw CSV file from DLD.
FINAL_OUTPUT_PATH	Path	D:\Database\...\1_july_to_12_aug_processed.xlsx	Final enriched Excel output file.
TEST_MERGE_PATH	Path	D:\Database\...\1_july_to_12_aug_test_merge.xlsx	Audit merge Excel file after Step A.
CITY_ID	Integer	15	Database identifier matching Dubai in dim_city.
CITY_NAME	String	Dubai	Case-sensitive city filter name.
db_params	Dict	host: localhost, port: 5432, database: nilesh...	PostgreSQL database credentials and target.

3. Pipeline Execution Workflow

STEP 1 & 1b: Load and Deduplicate

- Data Ingestion: The raw file is loaded using pandas, enforcing all data as strings (dtype=str) and reading with utf-8-sig encoding to remove any byte-order mark issues.
- Whitespace Trimming: Leading/trailing whitespace is stripped from all column names immediately.
- Strict Deduplication: Duplicates are dropped based on 22 specific columns. The first match is kept (keep='first'):

```
TRANSACTION_NUMBER, INSTANCE_DATE, GROUP_EN, PROCEDURE_EN, IS_OFFPLAN_EN, IS_FREE_HOLD_EN, USAGE_EN, AREA_EN,
 PROP_TYPE_EN, PROP_SB_TYPE_EN, TRANS_VALUE, PROCEDURE_AREA, ACTUAL_AREA, ROOMS_EN, PARKING, NEAREST_METRO_EN
, NEAREST_MALL_EN, NEAREST_LANDMARK_EN, TOTAL_BUYER, TOTAL_SELLER, MASTER_PROJECT_EN, PROJECT_EN
```

STEP 2: Date Parsing & Fiscal Quarters

The string INSTANCE_DATE is parsed into date-time format (%Y-%m-%d %H:%M:%S). From this, the script extracts the year and computes the quarter as a string formatted as Q{quarter}-{year} (e.g., Q3-2026). NaNs/NaTs are handled gracefully.

STEP 3: Property Type Normalization

Raw sub-types from PROP_SB_TYPE_EN are renamed to property_type_raw and mapped to a clean set of standard property types. Unmapped rows default to Others.

Mapped Core Type	Raw Property Sub-Types (from DLD PROP_SB_TYPE_EN)
Plot	Agricultural, Land
Flat	Flat, Government Housing, Hotel Apartment, Residential, Residential Flats, Studio
Villa	Residential / Attached Villas, Residential / Residential Villa, Residential / Villas, Stacked Townhouses, Villa
Office	Office
Shop	Shop, Shopping Mall, Show Rooms
Others	Airport, Building, Commercial, General Use, Hospital, Hotel, Industrial, Labor Camp, Warehouse, etc.

STEP 4, 4a & 4b: Column Renaming & Custom Mappings

- Column Translation: Standardizes names (e.g., TRANSACTION_NUMBER → document_number, AREA_EN → location_name, PROJECT_EN → project_name).
- Transaction Category: Maps Sales → Sale, Mortgage → Mortgages, and Gifts → Gifts.
- Unit Configuration: Maps ROOMS_EN descriptions to normalized structures (e.g. 1 B/R → 1Bhk, 2 B/R → 2Bhk, 3 B/R → 3Bhk, 4 B/R+ → >3Bhk, Studio/Single Room → Flat, Office/Shop as-is, and gym/hotel/store → Others).

STEP 5 & 6: Schema Alignment & Default Configurations

The pipeline adds all 70+ columns required by the database schema. Static default values are filled:

```
df['city_name'], df['state_name'], df['country_name'] = 'Dubai', 'Dubai', 'United Arab Emirates'
df['is_llm_processed'], df['is_manual_processed'] = 'No', 'No'
df['source_accessibility'], df['source_accessibility_way'] = 'Easy', 'Download'
df['data_type'], df['data_source'] = 'Registered Document', 'Dld'
```

4. Database Enrichment & Indexing Integration

The pipeline connects to the target PostgreSQL database to resolve geolocation coordinates, internal IDs, and project/location indexes.

STEP A: Database Project & Location Mapping

1. Connecting: Initiates connection to Postgres database using SQLAlchemy and retrieves active data from public.dim_project and public.dim_location where city_id = 15.
2. Key Generation: Constructs matching keys (lowercase, trimmed, and whitespace-normalized) on project name and location name.
3. Merging: Performs a left-merge to pull project coordinates, project IDs, location IDs, and location coordinates.
4. **Strict Index Rule**: The project index is only pulled when **BOTH** the project name and the location name matched on the same row. If either name is blank or missing, the index remains null.
5. Auditing: Intermediate records are written to TEST_MERGE_PATH for review before removing merge helper keys.

STEP B: Incremental NR Index Assignment

For records that do not match existing entries in dim_project, the script assigns unique, sequential index keys in the nrXXXX format:

1. **Sequence Init**: Queries public.transactions to extract the maximum number of existing index values matching ^nr(\d+) for CITY_ID = 15. The starting sequence is set to max_nr + 1.
2. **DB Mapping Retrieval**: Pulls a dictionary of historic (location_name, project_name, city_name) → index associations from transactions already registered in Postgres.
3. **Assignment Resolution**:
   - If the key is encountered multiple times in the *current file run*, it reuses the newly assigned index.
   - If the key matches a historic relationship in the *database mappings*, it reuses that index.
   - If it is a completely new combination, it assigns a new index nr{next_num}, registers it, and increments the sequence.
4. **Cleanup**: Replaces float indices (e.g. 1001.0 → 1001) and strips lingering suffixes (e.g. __dubai).

STEP C: Location Coordinate Fallback

For rows that failed to find coordinates through project matching, the script queries public.dim_location. If a row matches a known location name, its latitude and longitude are backfilled using the location-level coordinates.

5. Output Formatting & Verification

• Column Reordering: Columns are reordered to match the 70+ target database table schema fields exactly.
• Capitalization Normalization: A cosmetic processing function converts all text/string columns to **Title Case**.
• Saving Excel: The finalized dataset is saved to FINAL_OUTPUT_PATH (Excel format) without index numbers.